

**What are the steps?** Flow-oriented modeling provides an indication of how data objects are transformed by processing functions. Behavioral modeling depicts the states of the system and its classes and the impact of events on these states. Pattern-based modeling makes use of existing domain knowledge to facilitate requirements analysis. WebApp requirements models are especially adapted for the representation of content, interaction, function, and configuration-related requirements.

**What is the work product?** A wide array of text-based and diagrammatic forms may be chosen for the requirements model. Each of these representations provides a view of one or more of the model elements.

**How do I ensure that I've done it right?** Requirements modeling work products must be reviewed for correctness, completeness, and consistency. They must reflect the needs of all stakeholders and establish a foundation from which design can be conducted.

modeled, called *object-oriented analysis*, focuses on the definition of classes and the manner in which they collaborate with one another to effect customer requirements.

Although the analysis model that we propose in this book combines features of both approaches, software teams often choose one approach and exclude all representations from the other. The question is not which is best, but rather, what combination of representations will provide stakeholders with the best model of software requirements and the most effective bridge to software design.

## 7.2 FLOW-ORIENTED MODELING

Although data flow-oriented modeling is perceived as an outdated technique by some software engineers, it continues to be one of the most widely used requirements analysis notations in use today.<sup>1</sup> Although the *data flow diagram* (DFD) and related diagrams and information are not a formal part of UML, they can be used to complement UML diagrams and provide additional insight into system requirements and flow.



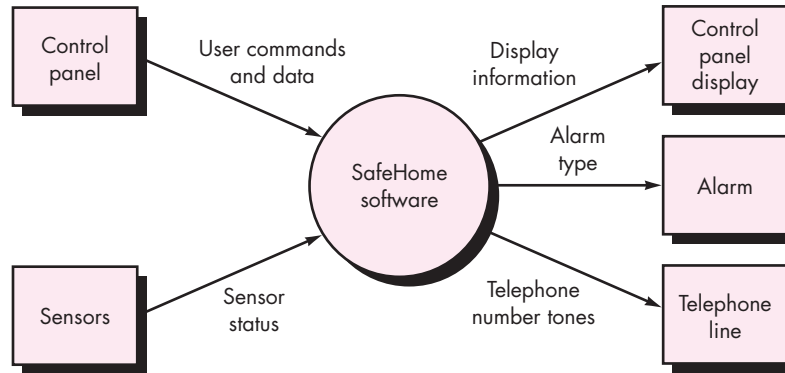
*Some will suggest that the DFD is old-school and it has no place in modern practice. That's a view that excludes a potentially useful mode of representation at the analysis level. If it can help, use the DFD.*

The DFD takes an input-process-output view of a system. That is, data objects flow into the software, are transformed by processing elements, and resultant data objects flow out of the software. Data objects are represented by labeled arrows, and transformations are represented by circles (also called bubbles). The DFD is presented in a hierarchical fashion. That is, the first data flow model (sometimes called a level 0 DFD or *context diagram*) represents the system as a whole. Subsequent data flow diagrams refine the context diagram, providing increasing detail with each subsequent level.

<sup>1</sup> Data flow modeling is a core modeling activity in *structured analysis*.

**FIGURE 7.1**

Context-level DFD for the *SafeHome* security function



### 7.2.1 Creating a Data Flow Model

#### note:

"The purpose of data flow diagrams is to provide a semantic bridge between users and systems developers."

Kenneth Kozar

The data flow diagram enables you to develop models of the information domain and functional domain. As the DFD is refined into greater levels of detail, you perform an implicit functional decomposition of the system. At the same time, the DFD refinement results in a corresponding refinement of data as it moves through the processes that embody the application.

A few simple guidelines can aid immeasurably during the derivation of a data flow diagram: (1) the level 0 data flow diagram should depict the software/system as a single bubble; (2) primary input and output should be carefully noted; (3) refinement should begin by isolating candidate processes, data objects, and data stores to be represented at the next level; (4) all arrows and bubbles should be labeled with meaningful names; (5) *information flow continuity* must be maintained from level to level,<sup>2</sup> and (6) one bubble at a time should be refined. There is a natural tendency to overcomplicate the data flow diagram. This occurs when you attempt to show too much detail too early or represent procedural aspects of the software in lieu of information flow.

To illustrate the use of the DFD and related notation, we again consider the *SafeHome* security function. A level 0 DFD for the security function is shown in Figure 7.1. The primary *external entities* (boxes) produce information for use by the system and consume information generated by the system. The labeled arrows represent data objects or data object hierarchies. For example, **user commands and data** encompasses all configuration commands, all activation/deactivation commands, all miscellaneous interactions, and all data that are entered to qualify or expand a command.

The level 0 DFD must now be expanded into a level 1 data flow model. But how do we proceed? Following an approach suggested in Chapter 6, you should apply a

<sup>2</sup> That is, the data objects that flow into the system or into any transformation at one level must be the same data objects (or their constituent parts) that flow into the transformation at a more refined level.

#### KEY POINT

Information flow continuity must be maintained as each DFD level is refined. This means that input and output at one level must be the same as input and output at a refined level.

“grammatical parse” [Abb83] to the use case narrative that describes the context-level bubble. That is, we isolate all nouns (and noun phrases) and verbs (and verb phrases) in a *SafeHome* processing narrative derived during the first requirements gathering meeting. Recalling the parsed processing narrative text presented in Section 6.5.1:



*The grammatical parse is not foolproof, but it can provide you with an excellent jump start, if you're struggling to define data objects and the transforms that operate on them.*

The *SafeHome security function* enables the *homeowner* to *configure* the *security system* when it is *installed*, *monitors* all *sensors* connected to the security system, and *interacts* with the homeowner through the *Internet*, a *PC*, or a *control panel*.

During *installation*, the *SafeHome PC* is used to *program* and *configure* the *system*. Each sensor is assigned a *number* and *type*, a *master password* is programmed for *arming* and *disarming* the system, and *telephone number(s)* are *input* for *dialing* when a *sensor event* occurs.

When a sensor event is *recognized*, the software *invokes* an *audible alarm* attached to the system. After a *delay time* that is specified by the homeowner during system configuration activities, the software dials a telephone number of a *monitoring service*, *provides information* about the *location*, *reporting* the nature of the event that has been detected. The telephone number will be *redialed* every 20 seconds until *telephone connection* is *obtained*.

The homeowner *receives security information* via a control panel, the PC, or a browser, collectively called an *interface*. The interface *displays prompting messages* and *system status information* on the control panel, the PC, or the browser window. Homeowner interaction takes the following form . . .



*Be certain that the processing narrative you intend to parse is written at the same level of abstraction throughout.*

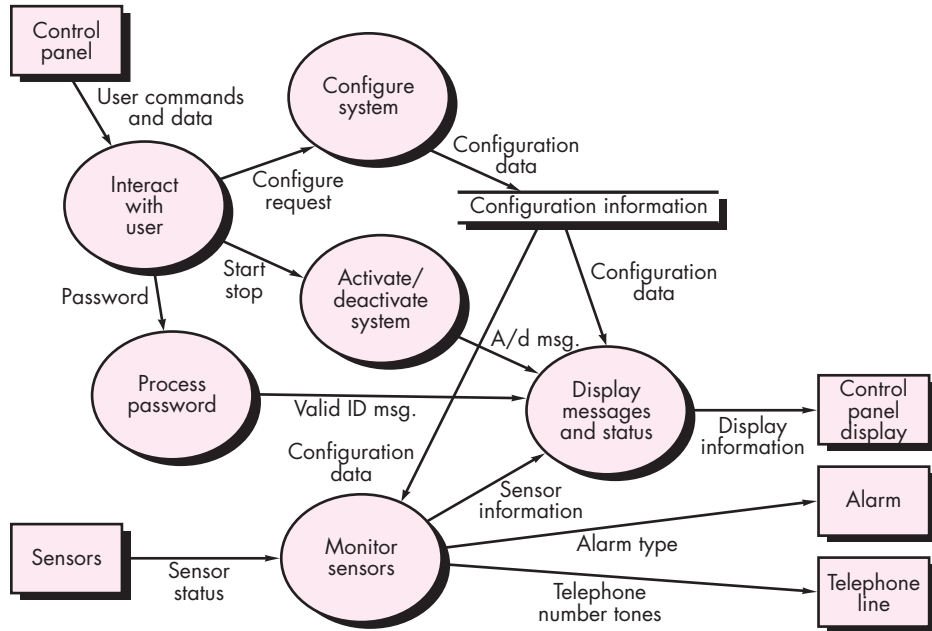
Referring to the grammatical parse, verbs are *SafeHome* processes and can be represented as bubbles in a subsequent DFD. Nouns are either external entities (boxes), data or control objects (arrows), or data stores (double lines). From the discussion in Chapter 6, recall that nouns and verbs can be associated with one another (e.g., each sensor is assigned a number and type; therefore **number** and **type** are attributes of the data object **sensor**). Therefore, by performing a grammatical parse on the processing narrative for a bubble at any DFD level, you can generate much useful information about how to proceed with the refinement to the next level. Using this information, a level 1 DFD is shown in Figure 7.2. The context level process shown in Figure 7.1 has been expanded into six processes derived from an examination of the grammatical parse. Similarly, the information flow between processes at level 1 has been derived from the parse. In addition, information flow continuity is maintained between levels 0 and 1.

The processes represented at DFD level 1 can be further refined into lower levels. For example, the process *monitor sensors* can be refined into a level 2 DFD as shown in Figure 7.3. Note once again that information flow continuity has been maintained between levels.

The refinement of DFDs continues until each bubble performs a simple function. That is, until the process represented by the bubble performs a function that would be easily implemented as a program component. In Chapter 8, I discuss a concept, called *cohesion*, that can be used to assess the processing focus of a given function. For now, we strive to refine DFDs until each bubble is “single-minded.”

**FIGURE 7.2**

Level 1 DFD for *SafeHome* security function

**FIGURE 7.3**

Level 2 DFD that refines the *monitor sensors* process

